

BAOS AI: Enterprise Technical Architecture & Decision Intelligence Guide (v2.0.0)

1. Project Foundation and Business Intelligence

1.1. Problem Landscape: The NP-Hard Challenge of Berth Allocation

Maritime berth allocation is a combinatorial optimization problem of **NP-hard complexity**. Harbor masters managing major ports face a high-stakes environment where dozens of vessels compete for limited berth space under rigid physical, temporal, and commercial constraints.

Current manual planning inefficiencies result in:

- **Suboptimal Utilization:** Baseline berth utilization typically plateaus at **40–60%**, significantly trailing the achievable target of **75–85%**.
- **Vessel Congestion:** Average wait times at congested ports range from **18 to 36 hours**.
- **High Operational Costs:** Idle vessels incur demurrage costs of **500–2,000 per hour**, direct losses to the charterer.
- **Contractual Penalties:** SLA violations for exceeding guaranteed wait times trigger penalties of **\$1,000+ per hour**.
- **Revenue Leakage:** Inefficient matching of vessel cargo to berth equipment leads to lost throughput and profit margins.

1.2. The BAOS AI Solution

BAOS AI is a full-stack AI and Constraint Optimization system that integrates Machine Learning predictions with Google’s CP-SAT solver to provide an automated, cost-aware decision intelligence platform.

Approach	Description	Key Limitations
Manual Planning	Assignments via experience, spreadsheets, and radio.	No optimization; high error rate; zero financial visibility.
Rule-Based	Static logic (e.g., First-Come-First-Served).	Cannot handle multi-objective trade-offs or complex learning.
Optimization-Driven	CP-SAT solver utilizing hard/soft constraints.	Deterministic; ignores historical uncertainty and data noise.
AI-Assisted (BAOS AI)	ML Predictions + CP-SAT + Uncertainty Engine.	Provides full decision intelligence with calibrated confidence.

1.3. Business Impact Assessment

The transition to BAOS AI delivers a proactive operational state evidenced by the following metrics:

Metric	Baseline (Before)	With BAOS AI
Berth Utilization	40–60%	75–85%
Average Vessel Wait	18–36 hours	4–12 hours
SLA Compliance	~70%	90–98%
Decision Transparency	None	Full Multi-Category Audit Trail
Risk Analysis	Manual Estimation	Real-time Monte Carlo Simulation

1.4. Evolutionary Path: Prototype to Production

The platform evolved from a Phase 1 Streamlit prototype to a Phase 2 production architecture (React/Next.js and FastAPI) to resolve critical infrastructure barriers.

Lessons Learned from Phase 1 Prototype:

- **Execution Model:** Streamlit reruns the entire script on interaction, preventing persistent state for complex undo/redo stacks.
- **Concurrency:** The single-threaded nature prevented multiple harbor masters from collaborating on a shared port schedule.
- **UI Rigidity:** Limited to standard widgets, preventing the development of high-density Gantt charts or interactive feasibility matrices.
- **Infrastructure:** Lack of built-in JWT authentication, persistent database ORM, and WebSocket support necessitated a full-stack migration.

2. High-Level System Architecture

2.1. Layered Architecture Overview

The system employs a modular engine-based architecture where every functional unit is isolated to ensure scalability and extensibility.

Layer	Engine / Component	Core Responsibility
Frontend	React / Next.js 16	UI Rendering, Zustand State, WebSocket client.
API	FastAPI (ASGI)	REST endpoints, Pydantic validation, JWT middleware.
Decision	decision_engine	ML-powered single-vessel ranking and confidence scoring.
Optimization	optimization_engine	CP-SAT model construction and constraint enforcement.
Commercial	commercial_engine	Revenue optimization and partnership tier management.

ML/Analytics	training_engine	Feature building, model training, and pattern discovery.
Simulation	simulation_engine	Event-driven "Digital Twin" and "What-If" conflict logic.
Data Layer	data_layer	Excel/JSON ingestion and SPEC/HISTORICAL provenance.

2.2. Request Lifecycle Analysis

Single Vessel Recommendation

1. **Ingestion:** User inputs vessel specs (LOA, Draft, Beam) into the UI.
2. **Inference:** decision_engine invokes Model 1 and Model 2 (GBR) to predict service/wait times.
3. **Classification:** Model 3 (RFC) determines probability of suitability across all berths.
4. **Confidence Scoring:** The 4-factor formula calculates risk based on data provenance and model variance.
5. **Delivery:** Top-K recommendations with structured reasoning are returned via API.

Multi-Vessel Optimization

1. **Parameterization:** User adjusts global weights (e.g., $w_{\{sla\}}$) via dashboard sliders.
2. **Model Build:** optimization_engine maps all 17 hard/soft constraints to the CP-SAT model.
3. **Solving:** Google OR-Tools generates an optimal schedule within a 30s timeout.
4. **Broadcast:** Results are pushed to all connected clients via Socket.IO.

2.3. Tech Stack Master List

Component	Library/Framework	Version	Rationale
Frontend	Next.js	16.2	Server-side rendering and file-based routing.
UI Core	React	19.2	Latest concurrent features and component lifecycle.
State	Zustand	5.0	High performance, minimal boilerplate state management.
Backend	FastAPI	0.110+	Async-native; 3–10x faster than Flask; Pydantic v2 support.
Optimizer	Google OR-Tools	9.9+	CP-SAT solver with native support for interval variables.
Database	PostgreSQL	14+	ACID compliance and robust JSONB column support.
Real-time	Socket.IO	4.x	Bi-directional communication with auto-reconnect.

3. Frontend Architecture (React/Next.js 16)

3.1. Framework Rationale

Next.js was implemented to replace Streamlit's script-rerun model with a React-native component lifecycle, enabling complex UI interactions, optimized build times, and secure JWT middleware for enterprise-grade authentication.

3.2. State Management with Zustand

The frontend is driven by three specialized stores (Auth, Optimizer, Simulation). To ensure data integrity, the system follows a **6-step State Synchronization Pattern**:

1. **Local Update:** UI reflects changes immediately (optimistic update).
2. **API Call:** Targeted POST request to the FastAPI optimization endpoint.
3. **Solver Execution:** CP-SAT re-calculates the global schedule with manual overrides locked.
4. **Response Processing:** Incoming JSON updates assignments, costs, and KPIs.
5. **Cross-component Consistency:** Timeline, matrix, and confidence panels sync simultaneously.
6. **Undo Stack:** Previous state is pushed to the stack for immediate recovery.

3.3. Design System & UI Components

The architecture utilizes Tailwind CSS with a custom property system for design tokens.

- **Key Classes:** `.card` (interactive hover lift), `.glass` (12px backdrop blur), `.btn-primary` (shadow lift), and `.heatmap-cell` (utilization-based color mapping).
- **Animation Triggers:** `fadeInUp` for initial page mounts and `pulse-glow` for elements actively being optimized.

3.4. Deep Dive: Optimizer Dashboard

- **Vessel Config Panel:** Dynamic form defining LOA, Beam, Draft, and DWT for N vessels.
- **Global Levers:** 10+ sliders to tune the objective function (e.g., `w_{waiting}` vs. `w_{commercial}`).
- **Interactive Gantt Timeline:** Hoverable chart distinguishing between wait (hatched) and service (solid) periods.
- **Feasibility Matrix:** A color-coded grid providing 6-factor analysis (LOA/Draft/Equipment/Cargo match).

3.5. Real-Time Communication

A **Singleton pattern** for the Socket.IO client ensures a single connection per lifecycle. The useSocket hook manages room-based subscriptions (e.g., `kpi:{port_id}`), enabling targeted data pushes without unnecessary global refreshes.

4. Backend Infrastructure & API Design (FastAPI)

4.1. Core Framework Features

FastAPI was selected over Flask due to its native async support (essential for I/O-bound database operations) and built-in OpenAPI documentation, which serves as the contract between the engine and the UI.

4.2. API Endpoint Registry

Method	Endpoint	Purpose
POST	/api/v1/optimize	Triggers multi-vessel CP-SAT optimization.
POST	/api/recommendations/get-recommendation	Generates ML-based berth rankings for one vessel.
POST	/api/v1/what-if	Compares baseline vs. modified scenario KPIs.
POST	/auth/login	Issues JWT access and refresh tokens.
POST	/auth/refresh	Silent rotation of expired access tokens.

4.3. Async Architecture

To maintain compatibility with Windows production environments and the asyncpg driver, the backend implements the SelectorEventLoop fix. The ASGI architecture allows the server to handle non-blocking database queries while offloading heavy CP-SAT solver tasks to separate threads.

4.4. Security & Authentication

- **JWT Architecture:** 24-hour access tokens (localStorage) and 7-day refresh tokens (DB-persisted for revocation).
 - **Encryption:** Passwords are never stored in plaintext, utilizing **bcrypt** with automatic salting.
 - **Refresh Flow:** The Axios interceptor detects 401 errors and attempts a silent refresh before retrying the user's initial request.
-

5. Database Architecture (PostgreSQL 14+)

5.1. Relational Design

PostgreSQL provides the ACID compliance required for financial cost tracking and the JSONB support needed for flexible vessel configurations and structured reasoning logs.

5.2. Entity Relationship Summary

The system utilizes **UUID primary keys** across all tables for distributed safety.

- **User:** Hashed credentials, role, and port_id.
- **Berth:** Max physical limits and **equipment_types JSON** (cranes, hoses).
- **Vessel:** Physical specs and **cargo_history JSON**.
- **Assignment:** Links vessels to berths; tracks net profit and turnaround hours.
- **Recommendation:** Stores **reasoning_json** and confidence score snapshots.

5.3. Commercial Intelligence Schema

Specialized tables drive the w_{commercial} weights in the optimizer:

- **berth_economics:** Columns: berth_class, specialization, rate_schedules, profit_margins, dynamic_pricing_thresholds.
- **vessel_companies:** Columns: tier (VIP/Premium/Standard), discount_percentage, annual_contract_value.
- **contract_obligations:** Columns: guaranteed_berth_types, priority_level, sla_turnaround_hours.

6. Machine Learning Pipeline & Analytics

6.1. Feature Engineering

Category	Features	Why It Matters
Vessel	LOA, Beam, Draft	Physical fit; Beam determines parallel berthing and fender needs.
Berth	Depth, Equipment Count	Direct eligibility and equipment-driven throughput potential.
Derived	Draft Clearance	Small clearance signals high tide sensitivity and operational risk.

6.2. Model Zoo

1. **Service Time Predictor:** GBR (Quantile Regression P25/P50/P75). **Hyperparameters:** 100 estimators, max_depth 5, learning_rate 0.1.
2. **Delay Predictor:** GBR modeling port congestion. **Hyperparameters:** 80 estimators, max_depth 4, learning_rate 0.1.

- 3. **Berth Suitability Classifier:** Random Forest Classifier (RFC). **Hyperparameters:** 300 estimators, no max_depth limit.
- 4. **Decision Ranker:** Score fusion logic penalizing variance (0.05 penalty if uncertainty > 0.5).
- 5. **XGBoost Ranker:** Ranks berths based on revenue optimization potential for commercial intelligence.
- 6. **Pattern Discovery Engine:** Unsupervised clustering to detect assignment anomalies.

6.3. Data Provenance & Confidence

Decisions are weighted by data source quality:

- **SPEC:** Verified (1.0).
- **HISTORICAL:** Partially verified (0.92).
- **ASSUMPTION:** Unverified/Default (0.80).

7. Constraint Optimization Engine (CP-SAT)

7.1. Mathematical Formulation

The solver minimizes a multi-objective cost function Z across all vessels V: $\min Z = \sum_{i \in V} (w_{\text{wait}} \cdot \text{Wait}_i + w_{\text{sla}} \cdot \text{Penalty}_i + w_{\text{comm}} \cdot \text{Cost}_i + w_{\text{dev}} \cdot \Delta_i)$ Where Wait_i is wait time, Penalty_i is SLA cost, Cost_i is commercial net loss, and Δ_i is the schedule deviation from the previous lock.

7.2. Constraint Library (C1-C17)

ID	Constraint	Logical Description	Implementation
C1	LOA Fit	$\text{loa}_v \leq \text{max_loa}_b$	Hard block if fit fails.
C2	Draft Clearance	$\text{draft}_v + \text{UKC} \leq \text{max_draft}_b$	Static check + tide adjustment.
C3	Beam Fit	$\text{beam}_v \leq \text{max_beam}_b$	Hard block if width exceeds limits.
C4	Non-Overlap	No concurrent vessels per berth.	AddNoOverlap(intervals_b)
C5	Berth Availability	Valid start/end within berth hours.	OnlyEnforceIf(X[v,b])
C6	Tide Window	Start must fall within safe water level.	Boolean selector enumeration.
C7	24x7 Flag	Night berthing restricted if flag=False.	Modular arithmetic on start time.

C8	Tug Capacity	Simultaneous movements $\leq$ tug count.	AddCumulative constraint.
C9	Pilot Capacity	Simultaneous movements $\leq$ pilot count.	AddCumulative constraint.
C10	Channel Limit	Max simultaneous channel movements.	Cumulative on inbound/outbound.
C11	Customs Clearance	Berthing blocked if uncustomed.	Hard block based on vessel status.
C12	FCFS Ordering	$\text{start}[v1] \leq \text{start}[v2]$ if $\text{eta}[v1] < \text{eta}[v2]$.	Pairwise ordering constraints.
C13	Gol Override	Government vessels prioritized first.	Priority-based pairwise constraints.
C14	Contract Pref.	Preferred berth bonus in objective.	$-w_{\text{contract}} \cdot X[v, \text{preferred_b}]$
C15	SLA Commitment	Penalty for wait > contract limit.	Linear penalty in objective.
C16	Downtime	Maintenance window exclusion.	Boolean disjunction ($\text{End} \leq \text{Start_DT}$).
C17	Weather	Global movement freeze for bad weather.	C16 applied to all berths.

7.3. Objective Function Configuration

The system supports **4-level weight granularity**, allowing specific combinations (e.g., Container Ships at Berth B1) to have custom wait-time penalties without affecting the global model.

8. Core Intelligence Engines Deep Dive

8.1. Cost Engine

Computes granular financial metrics:

- **Demurrage:** \$500/hr. **Fuel:** \$150/hr. **SLA Penalty:** \$1,000/hr.
- **Revenue:** \$2.50 per cargo ton (default).

8.2. KPI Engine

Tracks operational health vs. targets:

- **Berth Utilization:** Target 75–85%.
- **SLA Compliance:** Target >95%.
- **Throughput:** Total cargo tons per horizon hour.

8.3. Uncertainty Engine

Runs 200 Monte Carlo scenarios using:

- **ETA Uncertainty:** $\sigma=120$ min (Gaussian).
- **Weather Disruption:** $p=0.05$ (Bernoulli).
- **Equipment Failure:** $p=0.02$ per berth.
- **Risk Score:** $1 - CV$ (Coefficient of Variation). Scores >0.6 denote high fragility.

8.4. Simulation & Digital Twin

An event-driven simulation modeling vessel arrivals and departures chronologically. It powers the "What-If" analysis, propagating cascading delays when users manually reassign berths.

8.5. Explanation & Confidence Engines

- **Confidence Formula:** $\text{Score} = \alpha(S_{\text{feas}}) + \beta(S_{\text{pred}}) + \gamma(S_{\text{hist}}) + \delta(S_{\text{comp}})$.
- **Weights:** $\alpha=0.25$, $\beta=0.20$, $\gamma=0.20$, $\delta=0.35$.
- **Thresholds:** High Risk (<65%), Medium Risk (65–84%), Low Risk (≥85%).

9. Deployment, Limitations, and Strategic Roadmap

9.1. Current Deployment Stack

Next.js on Vercel/Docker; FastAPI with SelectorEventLoop (Windows fix) on the backend; PostgreSQL 14 for persistence.

9.2. Technical Debt & Limitations

1. **Stochastic Gap:** Current Monte Carlo runs are independent; lacks two-stage stochastic optimization.
2. **Synthetic Tides:** Semi-diurnal formulas used instead of live hydrographic API feeds.
3. **Real-Time Data:** No live AIS tracking or real-time port occupancy sensors.
4. **Resource Staticity:** Tugs/pilots are modeled as capacity counts, not individual schedules.
5. **RL Maturity:** The Reinforcement Learning weight agent is currently a non-functional stub.

9.3. Multi-Phase Roadmap

- **Short-Term (1-3 months):** AIS API integration, Celery for async solver jobs, and Bayesian confidence calibration.
- **Mid-Term (3-6 months):** Individual tug scheduling, real-time tide data, and multi-scenario comparison dashboards.
- **Long-Term (6-12 months):** RL-based autonomous planning and multi-port corridor scheduling.

9.4. Conclusion

BAOS AI v2.0.0 is a production-grade Decision Intelligence platform that bridges the gap between historical ML uncertainty and rigid physical constraints. By providing transparent reasoning and cost-aware optimization, it equips port authorities to maximize throughput in high-complexity maritime environments.